«Розробка та тестування C/C++ бібліотеки "ArrayStatistic"для статистичного аналізу»

(Завдання №25)

Виконав:
студент 2 курсу, групи комп'ютерної математики
Батрак К.В.

Зміст

1 Вступ **2**

2 Основна частина **2**

2.1 Архітектура та структура проекту 2

2.2 Реалізація модуля мовою C (`ArrayStatistic_C`) 3

2.2.1 Структура даних та керування пам'яттю 3

2.2.2 Реалізація основних операцій 3

2.3 Реалізація модуля мовою C++ (`ArrayStatistic_CPP`) 4

2.3.1 Структура класу та використання `std::vector` 4

2.3.2 Перевантаження операторів `+` та `*` 4

2.4 Система тестування та верифікації 5

2.4.1 Інтерактивне тестове меню 5

2.4.2 Крос-тестування C та C++ реалізацій 5

2.5 Теоретична база: Статистичні критерії 5

2.5.1 Коефіцієнт рангової кореляції Спірмена 5

2.5.2 Критерій рівності дисперсій (Фішера-Снедекера) 6

2.5.3 Критерії рівності середніх (t-тест та Z-тест) 6

2.5.4 Ранговий критерій Вілкоксона ("Вілсон") 6

3 Висновки **7**

1. Вступ

Сучасна обробка даних вимагає ефективних інструментів для зберігання та аналізу наборів інформації. Часто дані представлені не просто одновимірними масивами, а більш складними структурами, такими як масиви масивів або вектори векторів.

Метою даної роботи є розробка та тестування програмного модуля "ArrayStatistic"для зберігання та обробки масивів дійсних чисел, з реалізацією на мовах C та C++.

Для досягнення поставленої мети було визначено наступні **завдання**:

1. Спроектувати структуру даних для зберігання динамічного масиву масивів.
2. Реалізувати модуль мовою C (`ArrayStatistic.c`, `ArrayStatistic.h`), приділяючи особливу увагу ручному керуванню динамічною пам'яттю.
3. Реалізувати аналогічний за функціоналом модуль мовою C++ (`ArrayStatistic.cpp`, `ArrayStatistic.h` (C++)), використовуючи об'єктно-орієнтований підхід та інструменти Стандартної бібліотеки (STL).
4. Реалізувати повний набір операцій: додавання (масиву, елемента), видалення (за індексом, за значенням), об'єднання (+) та перетин (*).
5. Реалізувати набір ключових статистичних тестів для аналізу даних: коефіцієнт Спірмена, критерій Фішера, t-тест Стьюдента, Z-тест та критерій Вілкоксона.
6. Створити систему збірки на основі `Makefile` для обох проектів.
7. Розробити тестове середовище з інтерактивним меню (завантаження з файлу `ArrayStatistic` або з консолі) та виведенням результату у файл.
8. Створити окремий крос-тест (`ArrayStatistic_cross_test.cpp`) для верифікації того, що C та C++ реалізації дають ідентичні результати.

2. Основна частина

2.1. Архітектура та структура проекту

Відповідно до вимог, проект було розділено на дві головні директорії, що представляють два окремих проекти CLion, об'єднані однією системою контролю версій Git:

- `ArrayStatistic_C/`: Містить C-реалізацію.
- `ArrayStatistic_CPP/`: Містить C++ реалізацію.

Така структура дозволяє ізольовано розробляти та тестувати кожен модуль, але водночас використовувати їх разом у крос-тесті. Система збірки побудована на `Makefile`. Кожен проект має свій `Makefile`. `Makefile` C++ проекту є більш складним: він має цілі як для збірки власної тестової програми (`test_app_cpp`), так і для збірки крос-тесту

(`cross_test`). Ціль `cross_test` компілює C++ файли за допомогою `g++` та C-файли за допомогою `gcc`, після чого компонує отримані об'єктні файли (`.o`) в одну виконувану програму.

2.2. Реалізація модуля мовою C (`ArrayStatistic_C`)

2.2.1. Структура даних та керування пам'яттю

В основі C-реалізації лежить структура `ArrayStatistic`:

```
typedef struct {
    double** arrays;      //

    int* sizes;           //            ,

                                                  -
    int num_arrays;       //

    int capacity;         //

} ArrayStatistic;
```

Лістинг 1: Структура ArrayStatistic мовою C

Ключовою особливістю C-версії є **ручне керування пам'яттю**.

- **Створення** (`create_statistic`): Виділяє пам'ять для двох "керуючих"масивів: `arrays` (масив покажчиків `double*`) та `sizes` (масив `int`) за допомогою `malloc`.
- **Додавання** (`add_array`): Ця функція демонструє динамічне розширення. Перед додаванням нового масиву перевіряється, чи `num_arrays >= capacity`. Якщо так, ємність подвоюється, а керуючі масиви розширюються за допомогою `realloc`, зберігаючи існуючі дані. Важливо, що функція створює *внутрішню копію* масиву, що передається, використовуючи `malloc` та `memcpy`.
- **Звільнення** (`free_statistic`): Для уникнення витоків пам'яті, ця функція спершу проходить у циклі та звільняє кожен під-масив (`free(stats->arrays[i])`), і лише потім звільняє два керуючих масиви.

2.2.2. Реалізація основних операцій

- **Видалення** (`delete_array`, `delete_element`): Реалізовано за допомогою функції `memmove`. При видаленні масиву (або елемента), пам'ять спершу звільняється (`free`), а потім всі наступні елементи у керуючому масиві (або дані у під-масиві) зсуваються на одну позицію вліво, щоб "закрити дірку".

- **Об'єднання та Перетин** (`union_structures`, `intersection`): Ці функції вимагали створення допоміжних функцій `are_arrays_equal` (яка використовує `memcmp` для побайтового порівняння масивів) та `is_array_present` (яка шукає масив у структурі). `union_structures` створює нову структуру, копіює туди перший операнд, а потім додає з другого лише ті масиви, яких ще немає. `intersection` додає у нову структуру лише ті масиви, які присутні в обох операндах.

2.3. Реалізація модуля мовою C++ (`ArrayStatistic_CPP`)

2.3.1. Структура класу та використання `std::vector`

C++ реалізація демонструє потужність ООП та STL. Клас `ArrayStatistic` приховує всю складність керування пам'яттю за простим інтерфейсом.

```cpp
class ArrayStatistic {
private:
    std::vector<std::vector<double>> m_arrays;
    // ...                                                    ...
public:
    // ...                                  ...
};
```

Лістинг 2: Клас ArrayStatistic мовою C++

- **Керування пам'яттю**: Ручне керування пам'яттю повністю відсутнє. `std::vector` автоматично керує виділенням, розширенням та звільненням пам'яті (принцип RAII).
- **Додавання** (`addArray`, `addElement`): Реалізовано тривіально через `m_arrays.push_back(...` або `m_arrays[i].push_back(...)`. Вектор сам піклується про `realloc`.
- **Видалення** (`deleteArray`, `deleteElement`): Реалізовано через метод `.erase()`. Вектор автоматично виконує зсув елементів, що робить код значно чистішим та безпечнішим порівняно з `memmove` у C.

2.3.2. Перевантаження операторів `+` та `*`

Для реалізації об'єднання та перетину були перевантажені оператори `+` та `*`. Це дозволяє писати інтуїтивно зрозумілий код, наприклад `ArrayStatistic stats3 = stats1 + stats2;`.

Для реалізації цих операторів знадобився допоміжний метод `isArrayPresent`. На відміну від C-версії, де ми писали `memcmp`, у C++ достатньо використати `operator==` для `std::vector`, який автоматично порівнює і розміри, і вміст.

2.4. Система тестування та верифікації

2.4.1. Інтерактивне тестове меню

Обидві тестові програми (`ArrayStatistic_test.c`, `ArrayStatistic_test.cpp`) реалізують ідентичну логіку меню. При запуску програма запитує користувача, як завантажити дані:

1. **Завантажити з файлу** `ArrayStatistic_test.dat`: Програма відкриває файл, зчитує кількість масивів, а потім у циклі зчитує розмір кожного масиву та його елементи, додаючи їх до структури/класу.

2. **Ввести вручну**: Програма запитує користувача, скільки масивів він хоче ввести, а потім у циклі питає розмір та елементи кожного.

Після завантаження даних програма послідовно демонструє роботу *всіх* реалізованих функцій (видалення, операції `+`/`*`, всі статистичні тести) на завантажених даних. Наприкінці роботи результат зберігається у `results_C.txt` або `results_CPP.txt`.

2.4.2. Крос-тестування C та C++ реалізацій

Файл `ArrayStatistic_cross_test.cpp` є ключовим для верифікації проекту.

1. **Підключення:** Він підключає C++ клас `ArrayStatistic.h` та C-структуру `ArrayStatistic`

2. **Ізоляція:** Щоб уникнути конфлікту імен (оскільки і клас, і структура називаються `ArrayStatistic`), C-версія підключається всередині окремого `namespace C_Version` з використанням `extern "C"`.

3. **Тестування:** Програма `main` створює C++ об'єкт (`ArrayStatistic`) та C-структуру (`C_Version::ArrayStatistic`). Вона завантажує *однакові* дані з `ArrayStatistic_test.dat` в обидві сутності.

4. **Верифікація:** Викликаються всі статистичні функції для обох реалізацій, а їх результати порівнюються за допомогою допоміжної функції `are_equal`, яка враховує похибку обчислень `double` та `NAN`. Успішне проходження всіх тестів (вивід `PASSED`) доводить, що обидві реалізації є коректними та логічно ідентичними.

2.5. Теоретична база: Статистичні критерії

У модулях було реалізовано 5 статистичних методів для аналізу даних.

2.5.1. Коефіцієнт рангової кореляції Спірмена

- **Що це:** Непараметричний показник для вимірювання сили та напрямку зв'язку між двома змінними.

- **Сенс:** На відміну від коефіцієнта Пірсона, Спірмен оцінює не лінійний, а *монотонний* зв'язок. Він працює з *рангами* (порядковими номерами значень), а не самими значеннями. Це робить його стійким до викидів та ненормально розподілених даних.
- **Інтерпретація:** Значення від +1 (повний прямий монотонний зв'язок) до -1 (повний зворотний). 0 означає відсутність монотонного зв'язку.
- **Формула:** $\rho = 1 - \frac{6\sum d_i^2}{n(n^2-1)}$, де d_i — різниця між рангами i-го елемента у двох масивах, а n — кількість елементів.

2.5.2. Критерій рівності дисперсій (Фішера-Снедекера)

- **Що це:** F-тест, що використовується для перевірки нульової гіпотези про те, що дисперсії двох вибірок рівні.
- **Сенс:** Він відповідає на питання: "Чи є 'розкид' значень в одній групі значущо більшим, ніж в іншій?". Це часто є попередньою перевіркою для t-тесту.
- **Формула:** $F = \frac{s_1^2}{s_2^2}$, де s^2 — незміщена вибіркова дисперсія $\frac{\sum(x_i-\bar{x})^2}{n-1}$. Зазвичай, більша дисперсія ділиться на меншу.

2.5.3. Критерії рівності середніх (t-тест та Z-тест)

- **Що це:** Два тести для перевірки гіпотези про рівність середніх значень (μ) двох незалежних вибірок.
- **t-критерій Стьюдента** (наша `calculate_t_test_small`): Використовується, коли розміри вибірок малі (зазвичай $n < 30$) і дисперсія генеральної сукупності невідома. Він враховує невизначеність, пов'язану з малою вибіркою.
- **Z-критерій** (наша `calculate_z_test_large`): Використовується для великих вибірок (зазвичай $n \geq 30$). Згідно з центральною граничною теоремою, при великих n вибірковий розподіл середніх наближається до нормального, що дозволяє використовувати Z-статистику.

2.5.4. Ранговий критерій Вілкоксона ("Вілсон")

- **Що це:** Непараметричний критерій, також відомий як критерій Манна-Уітні (U-тест).
- **Сенс:** Це непараметрична альтернатива t-тесту. Він перевіряє нульову гіпотезу про те, що дві вибірки походять з одного й того ж розподілу (тобто, чи має одна вибірка тенденцію мати значення вищі або нижчі, ніж інша).
- **Логіка:** Обидві вибірки об'єднуються, сортуються, і кожному значенню присвоюється ранг. Потім обчислюється сума рангів для однієї з груп (напр., R_1).

- **Формула:** $U_1 = R_1 - \frac{n_1(n_1+1)}{2}$. Чим менше значення U, тим більш значуща різниця між вибірками.

3. Висновки

У ході виконання даної роботи було успішно досягнуто всіх поставлених цілей та завдань.

Було розроблено два програмних модулі мовами C та C++ для зберігання та обробки масивів масивів. C-реалізація (`ArrayStatistic.c`) продемонструвала фундаментальні аспекти програмування, зокрема ручне керування динамічною пам'яттю за допомогою `malloc`, `realloc` та `free`.

C++ реалізація (`ArrayStatistic.cpp`) продемонструвала переваги об'єктно-орієнтованого підходу та Стандартної бібліотеки (STL). Використання `std::vector` повністю абстрагувало керування пам'яттю (RAII), а перевантаження операторів `+` та `*` дозволило створити інтуїтивний інтерфейс для складних операцій об'єднання та перетину.

Було реалізовано повний набір статистичних критеріїв (Спірмен, Фішер, t-тест, Z-тест, Вілкоксон), що дозволяє використовувати розроблений модуль для повноцінного аналізу даних.

Коректність обох реалізацій була підтверджена за допомогою всебічного тестового оточення, що включало інтерактивне меню та зчитування з файлу даних, а також фінального крос-тестування (`ArrayStatistic_cross_test.cpp`), яке довело ідентичність результатів обох модулів.